Write D Flip-Flop and T Flip-Flop Using Verilog HDL Behavioural Modelling

Aim

To design and simulate a **D Flip-Flop and T Flip-Flop using Verilog HDL behavioural modelling** and verify its operation using Quartus Prime.**Objective**

After completing this experiment, students will be able to:

- To understand the working principle of a D Flip-Flop and T-Flip-Flop.
- To implement a D Flip-Flop and T-Flip-Flop using Verilog HDL.
- To understand behavioural modelling.
- To understand hold and toggle operations.
- To simulate the circuit using Quartus Prime.
- To verify the output using the truth table.

Theory:

PART A — D FLIP-FLOP

A **D Flip-Flop (Data Flip-Flop)** is a sequential circuit used to store one bit of information. It has one data input D, one clock input CLK, and outputs Q and $\overline{Q}$.

The input data is transferred to the output at the active edge of the clock. In this experiment, the **positive/rising edge** of the clock is used.

Characteristic Equation :

$$Q_{nesxt} = D$$

Block Diagram and Truth Table:-

D Flip-flop

Symbol

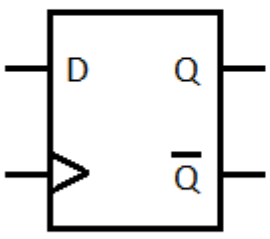


Table of truth:

clk	D	Q	$\overline{Q}$
0	0	Q	$\overline{Q}$
0	1	Q	$\overline{Q}$
1	0	0	1
1	1	1	0

Algorithm

- **Start.**
- **Declare D, CLK, Q and $\bar{Q}$.**
- **Use an always block triggered by the positive edge of the clock.**
- **Transfer the value of D to Q.**
- **Generate $\bar{Q}$ using the NOT operation.**
- **Create the testbench.**
- **Apply different values of D.**
- **Generate the clock signal.**
- **Simulate the design.**
- **Observe and verify the waveform.**
- **Stop.**

Verilog Code (Design Module)

```
module jk_flipflop (
input J,
input K,
input CLK,
input RESET,
output reg Q );
 always @(posedge CLK or posedge RESET)
begin
if (RESET)
Q <= 1'b0;
else
begin
case ({J, K})
2'b00: Q <= Q; // No change
2'b01: Q <= 1'b0; // Reset
2'b10: Q <= 1'b1; // Set
 2'b11: Q <= ~Q; // Toggle
endcase
end
end
endmodule
```

Testbench

```
module jk_flipflop_tb;

reg J;
   reg K;
   reg CLK;
   reg RESET;
   wire Q;

   // Instantiate the JK flip-flop
   jk_flipflop uut (
      .J(J),
      .K(K),
.CLK(CLK),
      .RESET(RESET),
      .Q(Q)
   );
   // Clock generation
   always #5 CLK =
   ~CLK;
   // Test sequence
   Initial begin
      // Initial values CLK = 1'b0;
      J = 1'b0; K = 1'b0; RESET = 1'b1;

      // Reset the flip-flop #10;
      RESET = 1'b0;

      // J=0, K=0 : No change
      J = 1'b0;K = 1'b0; #10;

      // J=0, K=1 : Reset
      J = 1'b0; K = 1'b1; #10;

      // J=1, K=0 : Set
      J = 1'b1;K = 1'b0; #10;

      // J=1, K=1 : Toggle
      J = 1'b1;K = 1'b1; #10;

      // Toggle again
      J = 1'b1;K = 1'b1; #10;

      // Stop simulation
 $stop;
 end
endmodule
```

Advantages of D Flip-Flop

1. **Simple circuit design.**
2. **Stores one bit of data.**
3. **No invalid input condition.**
4. **Useful for data storage and synchronization.**
5. **Easy to implement using Verilog HDL.**

Disadvantages of D Flip-Flop

1. **Stores only one bit of information.**
2. **Requires a clock signal for controlled operation.**
3. **Propagation delay is present.**
4. **Large storage systems require many flip-flops.**

Applications

- **Data storage.**
- **Registers.**
- **Shift registers.**
- **Data synchronization.**
- **Memory circuits.**
- **Pipeline registers.**
- **Digital counters and sequential circuits.**

PART A — T FLIP-FLOP

Theory

A **T (Toggle) Flip-Flop** is a sequential circuit that stores one bit of information.

It has one input T and a clock input CLK.

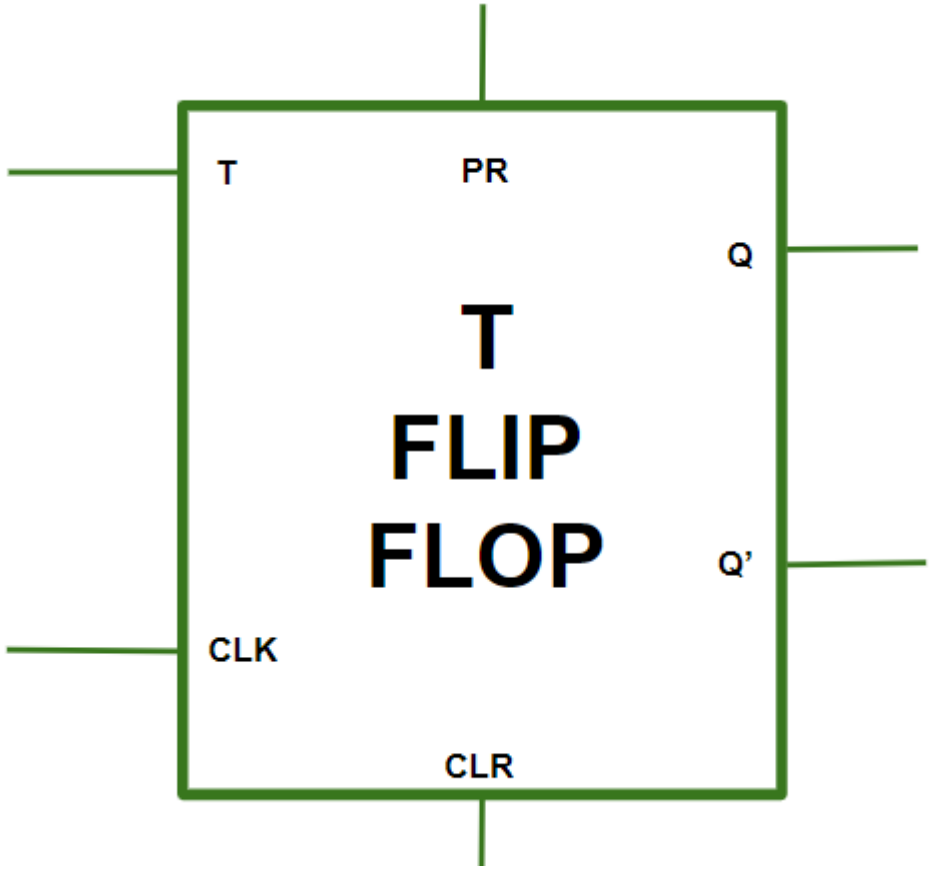
Its operation is:

- When **T = 0**, the output remains unchanged.
- When **T = 1**, the output toggles.

Characteristic Equation

$$Q_{nesxt} = T \oplus D$$

Block Diagram and Truth Table:-



Truth Table:

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

Algorithm

1. **Start.**
2. **Declare T, CLK, Q and $\bar{Q}$.**
3. **Use an always block triggered by the positive edge of the clock.**
4. **Check the value of T.**
5. **If T = 0, retain the previous value of Q.**
6. **If T = 1, toggle Q.**
7. **Generate $\bar{Q}$.**

8. **Create the testbench.**
9. **Apply different values of T.**
10. **Generate the clock.**
11. **Simulate the design.**
12. **Observe and verify the waveform.**
13. **Stop.**

Verilog Code (Design Module)

```
module
   t_flipflop (
   input T,
   input
   CLK,
   input
   RESET
   , output
   reg Q
);

always @(posedge CLK or posedge
RESET) begin
   if (RESET) Q <= 1'b0;
   else if (T ==
      1'b0) Q
      <= Q;
   else
      Q <= ~Q;

end

 endmodule
```

Testbench

module

t_flipflop_tb;

reg T;

reg

```
CLK;
reg
RES
ET;
wire
Q;

t_flipflop uut (
    .T(T),

    .CLK(CLK),

    .RESET(RESET),

    .Q(Q)

);


always #5 CLK =

~CLK; initial
begin
    CLK = 0;

    T = 0;RESET = 1; #10

    RESET = 0;

    #10  T = 0;  // No change

    #10 T = 1;  // Toggle
    #10 T = 1;  // Toggle
    #10 T = 0;  // No change
     #10 T = 1;  // Toggle

    #10 $stop;
    end

endmodule
```

Simulation Procedure (Quartus Prime & QuestaSim)

Step 1: Create Project

Open **Quartus Prime → File → New Project Wizard**.

Enter the project name and select the required FPGA device.

Step 2: Create Verilog File

Go to:

File → New → Verilog HDL File

Enter the D or T flip-flop code and save it as:

Step 3: Create Testbench

Create another Verilog file:

nter the corresponding testbench code.

Step 4: Add Files

Go to:

Project → Add/Remove Files in Project

Add the design and testbench files.

Step 5: Set Top-Level Entity

For compilation, set:

as the top-level entity.

Step 6: Compile

Select:

Processing → Start Compilation

Check that the design compiles without errors.

Step 7: Open Simulator

Open the simulator configured with Quartus Prime.

Select the appropriate testbench as the simulation top-level.

Step 8: Add Signals

For D Flip-Flop:

Step 9: Run Simulation

Run the simulation for approximately **100 ns**.

Observe the waveform.

Step 10: Verify

For D Flip-Flop:

At every rising edge, Q follows D.

For T Flip-Flop:

At every rising edge, T = 0 holds Q and T = 1 toggles Q.

Result

The D Flip-Flop and T Flip-Flop were successfully designed and simulated using Verilog HDL behavioural modelling in Quartus Prime. The obtained waveforms were verified with their respective truth tables.

12. Conclusion

The experiment demonstrates the implementation of basic sequential circuits using Verilog behavioural modelling. The D Flip-Flop is mainly used for data storage, whereas the T Flip-Flop is mainly used for toggle operations, counters, and frequency division.

Viva Questions

1. What is a flip-flop?
2. What is behavioural modelling in Verilog?
3. What is the function of posedge?
4. What is the characteristic equation of D flip-flop?
5. What is the characteristic equation of T flip-flop?
6. What happens when T = 0?
7. What happens when T = 1?
8. What is the difference between D and T flip-flops?
9. Why is q declared as reg?
10. What is the purpose of the testbench?